Relatório técnico

Separação de responsabilidades, SOLID e validação GameStore | Lucas Oliveira | 04 de novembro de 2026

1. Abordagem adotada

Foi utilizada a alternativa de novo projeto prevista na observação da atividade. Como o sistema anterior não foi fornecido, não é possível documentar defeitos ou alterações reais daquele código. A implementação entregue já organiza as responsabilidades para permitir reaproveitamento em uma interface web futura.

2. Responsabilidades das classes

Classes	Responsabilidade
Product e Sale	Representar dados de domínio e proteger invariantes locais. Sale.total calcula o total com precisão decimal.
StoreState	Representar o estado agregado e fornecer listas imutáveis.
StoreService	Executar casos de uso: add, update, remove, sell, find, products e sales. Validar duplicidade e regras de estoque.
StoreRepository	Contrato mínimo para ler e salvar o estado completo.
FileRepository	Codificar, recuperar e gravar o estado em arquivo, sem regras de apresentação.
MemoryRepository	Fornecer implementação substituível em memória para testes.
ConsoleUI	Ler entradas e apresentar resultados e erros ao operador.
Main e SelfTest	Compor dependências e iniciar a aplicação; executar os testes por meio do main.
BusinessException	Distinguir violações de regras de negócio das falhas de entrada e saída.

3. Organização para reutilização

ConsoleUI depende do serviço; StoreService depende do contrato StoreRepository. FileRepository e MemoryRepository implementam esse contrato. Nenhuma entidade, serviço ou repositório importa Swing ou lê entrada do usuário. Uma futura camada HTTP pode chamar os mesmos casos de uso.

4. Princípios SOLID aplicados

Princípio	Aplicação e justificativa
S — Responsabilidade única	ConsoleUI apresenta; StoreService orquestra regras; FileRepository persiste; Product e Sale protegem seus próprios dados. Mudanças de armazenamento não exigem editar a interface.
O — Aberto/fechado	Um novo adaptador pode implementar StoreRepository sem alterar os casos de uso. A composição em Main escolhe a implementação.
L — Substituição de Liskov	FileRepository e MemoryRepository cumprem load/save. O mesmo serviço funciona com ambas; os testes cobrem cadastro e venda em memória e no arquivo.
I — Segregação de interfaces	O contrato StoreRepository possui somente load e save. Não exige métodos de tela, relatório, conexão ou funções que seus adaptadores não utilizam.
D — Inversão de dependência	StoreService recebe StoreRepository e Clock por construtor. Não cria diretamente armazenamento ou relógio de produção; isso permite substituir ambos nos testes.

5. Organização e refatorações estruturais

A separação foi realizada durante a construção do projeto novo; não é apresentada como comparação com código anterior. As decisões abaixo eliminam acoplamentos e duplicações típicos de uma classe que mistura interface, regras e armazenamento.

Extração de responsabilidade: leitura e mensagens estão em ConsoleUI; validação e casos de uso em StoreService; acesso ao arquivo em FileRepository. Isso evita uma classe concentrando todas as funções.

Encapsulamento do estado: StoreState e os records Product/Sale substituem coleções e campos mutáveis expostos. A operação sell cria um novo estado, sem permitir estoque parcial ou histórico atualizado isoladamente.

Centralização de validações: Product verifica nome, preço e estoque; uniqueName e find evitam repetição de buscas e verificação de duplicidade. commit centraliza a ordem salvar-depois-publicar, evitando alterar a memória antes da gravação.

Precisão e constantes: BigDecimal elimina aritmética monetária com double; MAGIC identifica a versão do arquivo; Clock substitui chamadas diretas ao relógio do sistema para permitir testes repetíveis.

6. Padrões empregados

Repository: StoreRepository abstrai o armazenamento do estado. Seus adaptadores em arquivo e memória permitem reutilização e testes sem banco externo. A granularidade do estado completo foi escolhida para salvar estoque e histórico juntos.

Service Layer: StoreService oferece os casos de uso para qualquer apresentação. Injeção de dependências por construtor define o repositório e o relógio. Main atua como ponto de composição. Não foi introduzido framework de injeção ou Singleton.

7. Validação executada

Compilação de todos os fontes com Java 17, UTF-8, --release 17, -Xlint:all e -Werror, sem avisos. Geração do JAR com Main-Class e execução real de Main --test: 26 verificações aprovadas. A saída integral está em evidencias/testes-main.txt.

Cobertura: cadastro; normalização; duplicidade; preço zero, negativo e fracionário inválido; estoque negativo; identificador inexistente; quantidades inválidas; estoque insuficiente; cálculo monetário; baixa de estoque; relógio fixo; alteração; listas imutáveis; exclusão; preservação de histórico; persistência em disco; acentos; arquivo corrompido; falha de gravação e conservação do estado.

Teste adicional do JAR em três processos distintos confirmou cadastro, recuperação após reinício, venda, consulta, alteração e exclusão. Saída em evidencias/teste-console.txt. Os dados de teste foram isolados em pastas temporárias. A interface gráfica do NetBeans não foi executada neste ambiente; a compilação e a execução Java foram verificadas diretamente.

8. Repositório e evidências

Repositório público publicado: https://github.com/LucasSilva-DT/UC15-PI. Branch principal: main. O código e os resultados dos testes foram publicados no commit e15d43cbce9dbc420cf1cb2fdc94f6468ae667a9, em 04/11/2026 às 15:51:47 (Brasília).

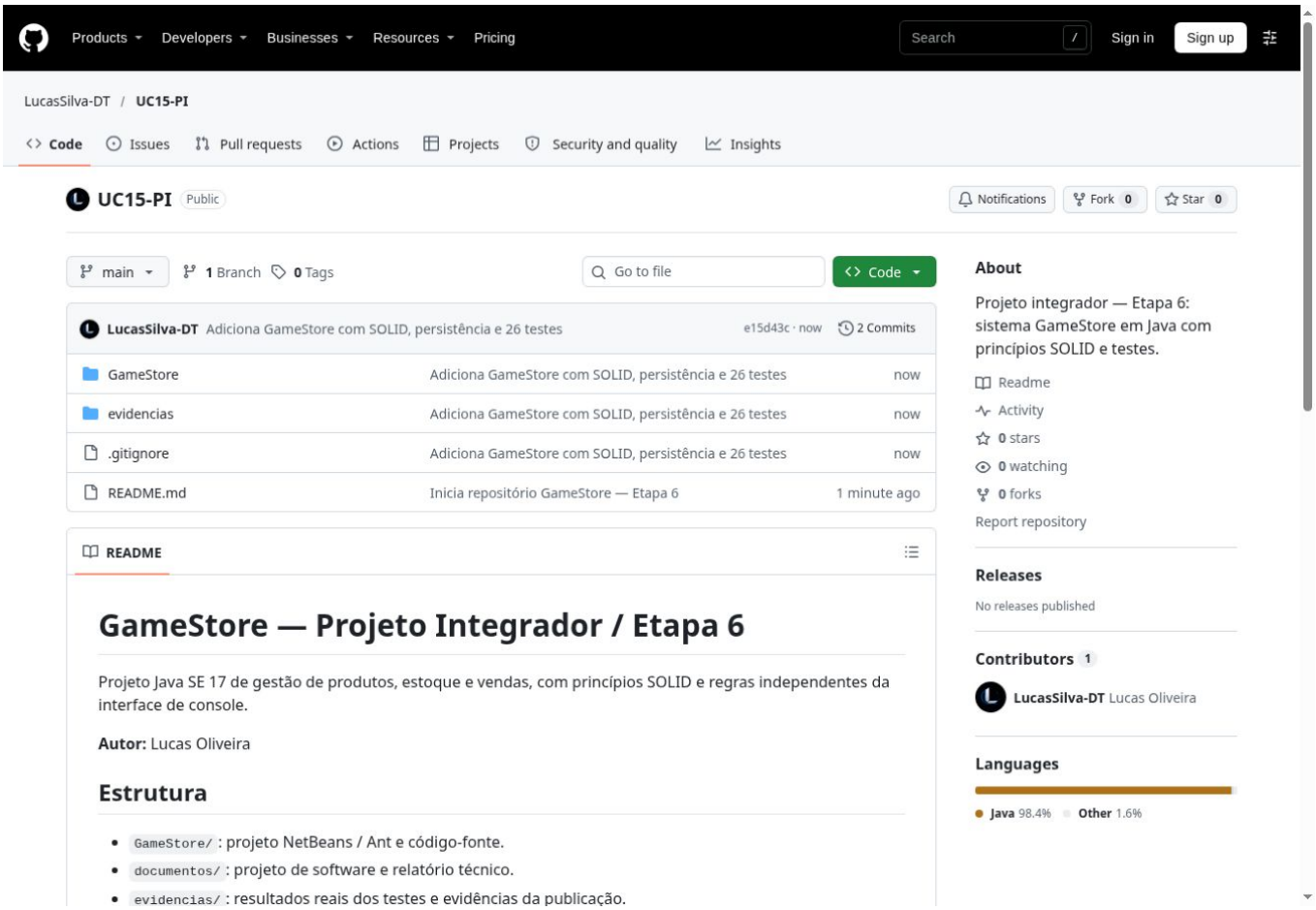
As páginas seguintes apresentam capturas reais do repositório e do commit de publicação do código. Os arquivos originais estão em evidencias/github-repositorio.jpg e evidencias/github-commit.jpg; github-commit.json registra metadados verificados pela API. Os documentos e estas evidências são adicionados em commit posterior.

9. Conclusão técnica

O projeto entregue implementa o escopo funcional documentado, separa regras da interface e apresenta verificações executadas e evidências reais do GitHub. Foi adotado o projeto novo permitido no enunciado, sem atribuir alterações a um sistema anterior não disponibilizado.

10. Evidência do repositório publicado

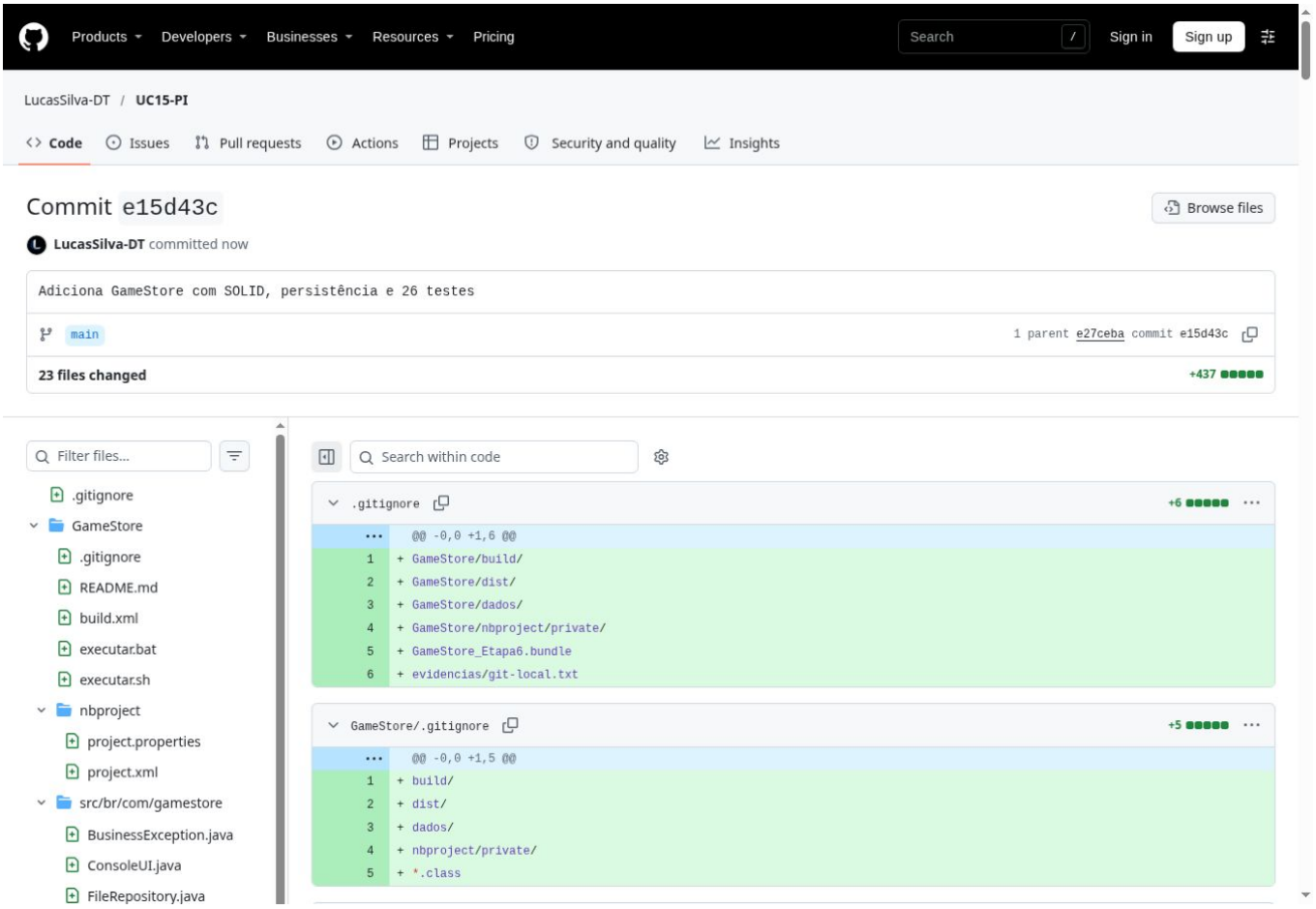
Página pública do projeto após publicação do código. Repositório: LucasSilva-DT/UC15-PI. A captura mostra a pasta GameStore, os resultados de testes, o README e a indicação do commit e15d43c.



Abrir repositório no GitHub

11. Evidência do commit de publicação

Commit e15d43cbce9dbc420cf1cb2fdc94f6468ae667a9. Mensagem: Adiciona GameStore com SOLID, persistência e 26 testes. A captura mostra a autoria, o identificador, os arquivos alterados e os fontes Java na árvore.



Abrir commit de publicação no GitHub